



KIET
GROUP OF INSTITUTIONS
Connecting Life with Learning



A
Project Report
on
ASKLMS – A Unified Learning & Institute
Management Platform
submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE

SESSION 2025-26
in
Computer Science and Engineering

By
Ananya Jain (2200290100020)
Srijan Omar (2200290130165)

Under the supervision of
Mr. Vipin Deval

KIET Group of Institutions, Ghaziabad

Affiliated to
Dr. A.P.J. Abdul Kalam Technical University, Lucknow
(Formerly UPTU)
May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name: Ananya Jain

Roll No.: 2200290100020

Signature:

Name: Srijan Omar

Roll No.: 2200290130165ⁱ

CERTIFICATE

This is to certify that Project Report entitled “AskLMS – A Unified Learning & Institute Management Platform” which is submitted by Ananya Jain in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science & Engineering of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

.

Mr. Vipin Deval

(Project Supervisor)

Date:

Dr. Vineet Sharma

(Dean CSE)

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Mr. Vipin Deval, Department of Computer Science & Engineering, KIET, Ghaziabad, for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Vineet Sharma, Dean of the Department of Computer Science & Engineering, KIET, Ghaziabad, for his full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members, especially faculty/industry person/any person, of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Signature:

Name: Ananya Jain

Roll No.: 2200290100020

Signature:

Name: Srijan Omar

Roll No.: 2200290130165

ABSTRACT

In recent years, coaching institutes and learning centers have become essential hubs for students preparing for academic and competitive examinations. However, most small and medium-sized coaching institutes continue to rely on manual methods for managing batches, collecting fees, distributing notes, and tracking student progress. ASKLMS is a robust batch management and learning management system designed to digitalize and automate the day-to-day operations of coaching institutes. The system integrates batch management, fee collection, student onboarding, and content delivery into a single platform.

TABLE OF CONTENTS

Page No.

DECLARATION.....	ii
CERTIFICATE.....	iii
ACKNOWLEDGEMENTS.....	iv
ABSTRACT.....	v
LIST OF FIGURES.....	viii
LIST OF ABBREVIATIONS.....	ix
CHAPTER 1 (INTRODUCTION).....	1
1.1. Introduction.....	1
1.2. Project Description.....	1
1.3. Background and Context of Coaching Institutes in India.....	2
1.4. Problems with Manual Coaching Institute Management.....	4
1.5. Need for an Integrated Digital Platform.....	5
1.6. Motivation of the project.....	6
1.7. Problem Statement.....	7
1.8. Objectives of the Project.....	7
1.9. Scope of the Project.....	8
1.10. Technologies Used.....	10
CHAPTER 2 (LITERATURE RIVIEW).....	12
CHAPTER 3 (PROPOSED METHODOLOGY)	14
3.1. Overview of Proposed System.....	14

3.2 System Architecture.....	18
3.3 Working Methodology.....	19
CHAPTER 4 (RESULTS AND DISCUSSION)	23
CHAPTER 5 (CONCLUSIONS AND FUTURE SCOPE).....	25
5.1. Conclusion.....	25
5.2. Key Achievemnets.....	26
5.3. Future Scope.....	27
REFERENCES.....	30
APPENDEX1.....	33

LIST OF FIGURES

Figure No.	Description	Page No.
1.1	Admin Dashboard	9
1.2	Student Dashboard	9
1.3	Technologies Used	12
2.1	Landing Page	13
2.2	Features of AskLms	13
3.1	Home Page	17
3.2	Role Base Access Control	17

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Full Form
1.	API	Application Programming Interface
2.	ASKLMS	Ask Learning Management System
3.	CDN	Content Delivery Network
4.	CORS	Cross-Origin Resource Sharing
5.	CRUD	Create, Read, Update, Delete
6.	CSS	Cascading Style Sheets
7.	DFD	Data Flow Diagram
8.	DRF	Django REST Framework
9.	ER	Entity-Relationship
10.	HMAC	Hash-based Message Authentication Code
11.	HTML	HyperText Markup Language
12.	HTTP / HTTPS	HyperText Transfer Protocol / Secure
13.	JEE	Joint Entrance Examination
14.	JSON	JavaScript Object Notation
15.	JSX	JavaScript XML
16.	LMS	Learning Management System
17.	MVC	Model-View-Controller
18.	MVP	Minimum Viable Product
19.	NEET	National Eligibility cum Entrance Test
20.	OAuth	Open Authorization
21.	ORM	Object-Relational Mapping

22.	PWA	Progressive Web Application
23.	RBAC	Role-Based Access Control
24.	REST	Representational State Transfer
25.	SDG	Sustainable Development Goal (United Nations)
26.	SDLC	Software Development Life Cycle
27.	SMTP	Simple Mail Transfer Protocol
28.	SQL	Structured Query Language
29.	SRS	Software Requirements Specification
30.	UAT	User Acceptance Testing
31.	UI / UX	User Interface / User Experience
32.	URL	Uniform Resource Locator
33.	VM	Virtual Machine
34.	WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction

In the present digital era, educational institutions are rapidly adopting technology-driven solutions to enhance efficiency, transparency, and accessibility. While universities and large organizations have embraced digital learning platforms, many small and medium-sized coaching institutes still rely on traditional offline methods for managing academic and administrative activities. These methods include manual batch records, handwritten attendance, offline fee collection, and physical distribution of study material. Such practices often lead to operational inefficiencies, data redundancy, lack of transparency, and difficulty in tracking student progress. With the growing number of students and batches, managing these processes manually becomes time-consuming and error-prone. Additionally, students today expect seamless digital access to learning resources, notifications, and payment facilities. To address these challenges, ASKLMS (Learning Management System) has been designed as an offline-oriented batch management and learning management system. The system aims to digitalize the core operations of coaching institutes while maintaining simplicity, affordability, and scalability. It bridges the gap between traditional offline coaching management and modern digital learning platforms.

1.2 Project Description

ASKLMS is a comprehensive web-based application designed specifically for coaching institutes to manage batches, students, fees, and learning content through a centralized platform. The system provides separate portals for students and administrators, ensuring role-based access and secure handling of data. The student portal allows learners to enroll in courses, pay monthly fees, download notes, access recorded lectures, and receive important notifications. The admin panel enables institute staff to manage courses, batches, fee

approvals, notes, reports, and student achievements efficiently. The system is developed using modern web technologies such as React for the frontend, Django REST Framework for the backend, and MySQL as the database. Docker-based deployment ensures scalability and ease of maintenance. The project is currently being deployed and tested at Nitant Academy, Saharanpur, serving as a real-world pilot implementation. ASKLMS is a comprehensive web-based application designed specifically for coaching institutes to manage batches, students, fees, and learning content through a centralized platform. The system provides separate portals for students and administrators, ensuring role-based access and secure handling of data. The student portal allows learners to enroll in courses, pay monthly fees, download notes, access recorded lectures, and receive important notifications. The admin panel enables institute staff to manage courses, batches, fee approvals, notes, reports, and student achievements efficiently. The system is developed using modern web technologies such as React for the frontend, Django REST Framework for the backend, and MySQL as the database. Docker-based deployment ensures scalability and ease of maintenance. The project is currently being deployed and tested at Nitant Academy, Saharanpur, serving as a real-world pilot implementation.

1.3 Background and Context of Coaching Institutes in India

The coaching institute sector in India is one of the largest and most dynamic informal education markets in the world. Industry analyses estimate the private tutoring and coaching market at approximately USD 40 billion as of 2023, with compound annual growth projected between 12 and 14 percent through 2030. Cities such as Kota in Rajasthan, Saharanpur and Ghaziabad in Uttar Pradesh, Patna in Bihar, and Hyderabad in Telangana collectively host tens of thousands of coaching centres that prepare millions of students annually for the Joint Entrance Examination (JEE) for engineering admissions, the National Eligibility cum Entrance Test (NEET) for medical admissions, and board assessments for Classes X and XII.

The educational significance of this sector is considerable. For a large proportion of students from Tier-II and Tier-III cities, coaching institutes provide the only structured preparation available beyond under-resourced government schools. The quality of instruction, the access

to study materials, and the consistency of fee-based enrollment create a de facto second tier of formal education. The economic significance is equally important: a thriving coaching institute provides livelihoods for the institute owner, faculty, and administrative staff, and often represents a family's principal investment in a child's educational future.

Yet despite this scale and significance, the operational infrastructure of the overwhelming majority of coaching institutes, particularly small and medium-sized establishments outside metropolitan areas, remains strikingly underdeveloped from a technology perspective. The structural reasons for this are multiple. Institute owners frequently have strong pedagogical expertise but limited technical background. Budget constraints make enterprise software platforms inaccessible. Existing LMS products designed for universities or corporate training are poorly matched to the specific operational patterns of an offline coaching centre, which involves monthly cash-based fee cycles, batch-level teaching rather than self-paced online learning, and a student base whose primary engagement is in-person rather than digital.

The COVID-19 pandemic of 2020 to 2022 created a forced and compressed experiment in digital education delivery. Large coaching chains with capital deployed proprietary mobile applications and live-streaming infrastructure. Smaller institutes attempted to use general-purpose tools, primarily Zoom for live lectures, WhatsApp groups for notes distribution, and Excel spreadsheets for fee tracking. These ad hoc arrangements, while providing temporary continuity, generated new problems including data inconsistency across spreadsheets maintained by different staff members, security vulnerabilities in WhatsApp-based content distribution, and the near-total absence of any unified view of institutional operations. When physical classes resumed, most smaller institutes reverted to pre-pandemic manual methods, having gained no lasting digital capability. The technology gap between large coaching chains and smaller community-based institutes, rather than narrowing, widened during this period.

ASKLMS is designed to address this gap directly. It is a production-grade, full-stack web application built from its foundations to serve the specific operational requirements of offline coaching institutes. It provides a unified digital platform for all core institute functions and is

designed to be deployable without enterprise-level infrastructure budget or specialised technical staff.

1.4 Problems with Manual Coaching Institute Management

The reliance on manual administrative processes in coaching institutes generates a cluster of persistent and interconnected operational problems. Understanding these problems in their specific context is necessary to appreciate the design decisions made in ASKLMS.

Fee management is the most acute operational challenge. In a typical offline coaching institute, each student pays a monthly tuition fee that may vary by batch, by subject combination, or by the application of discounts for merit, family hardship, or sibling enrollment. Manual fee collection requires physical presence at the institute's counter. Receipts are issued in handwritten duplicate books. Reconciliation at the end of each month requires matching each receipt stub against a manually maintained ledger, a process that routinely takes staff several hours and still yields discrepancies. There is no systematic mechanism for identifying overdue accounts, sending reminders, or generating consolidated revenue reports without significant manual effort.

Batch administration presents a related set of challenges. An institute offering five to ten batches concurrently must track each batch's composition, capacity utilisation, assigned faculty, schedule, and the month from which each student starts paying. Mid-session student transfers between batches, faculty substitutions, and batch capacity adjustments are managed through informal communication and handwritten amendments to registers. The absence of a digital system means there is no reliable audit trail for changes and no automatic propagation of schedule updates to affected students.

Content delivery is physically constrained. Printed and photocopied study materials involve production costs, distribution logistics, and an absence of version control. A faculty member who updates a study sheet after an initial distribution cannot easily ensure that all students receive the updated version. Recorded lectures, increasingly expected by students preparing

for competitive examinations who wish to revise at their own pace, have no supported delivery mechanism in a paper-based institute.

Communication with students and parents relies on high-effort, low-reliability channels. Phone calls to individual families are time-consuming and undocumented. WhatsApp broadcasts lack delivery confirmation, cannot be segmented by batch, and cannot be correlated with fee or schedule records. The absence of a structured notification system means that reminders for fee due dates, examination schedules, and class cancellations frequently fail to reach all affected students in time.

Progress tracking and reporting are similarly impeded. Without a digital record of attendance, assessment scores, and content access, institute administrators cannot generate the kind of aggregate analytics that would allow them to identify at-risk students, evaluate faculty performance, or demonstrate outcomes to parents. Management reporting is correspondingly shallow.

1.5 Need for an Integrated Digital Platform

The problems described in Section 1.2 collectively make the case for a purpose-built integrated digital platform for coaching institutes. The emphasis on integration is critical: individual point solutions address individual problems but do not resolve the underlying issue, which is the absence of a unified data model connecting student identity, batch enrollment, fee records, content access, and communication history.

A student management module in isolation can record enrollment but cannot link enrollment records to fee obligations or content access rights. A payment processing module in isolation can record transactions but cannot automatically determine which students are overdue based on their batch enrollment date and fee schedule. A content delivery module in isolation can serve files but cannot restrict access based on whether a student's fees are current. Integration is not an implementation convenience; it is the source of the system's operational value.

The specific requirement in the Indian coaching institute context is for a platform that is, first, affordable to deploy and operate. Enterprise LMS platforms such as Blackboard, Canvas, and

commercial ERP products for education are priced for institutional budgets that most small institutes do not have. The platform must be self-hostable, or at minimum available as a low-cost subscription, with infrastructure costs within the financial reach of a business generating monthly revenue of one to five lakh rupees.

Second, the platform must be operable without specialised technical staff. Institute owners and administrators are educators and managers, not software engineers. The operational interface must be intuitive enough that a staff member with general computer literacy can be trained to perform all standard functions within a few hours. Third, the platform must accommodate the offline-first operational pattern of coaching institutes. Batch students attend in-person classes. Fee collection has historically been cash-based. The platform must handle offline payments alongside online transactions, not assume that all operations are digital.

Fourth, the platform must be extensible to multiple institutes over time. A successful platform serving a single coaching centre must be architected in a way that allows it to serve multiple institutes, each with their own student population, batch structure, and branding, without requiring separate deployments for each client. ASKLMS addresses all four requirements through its multi-tenant-ready architecture, configurable permission system, and Docker-based deployment model.

1.6 Motivation of the Project

The motivation for this project arose from a direct and concrete experience. During the third year of the B.Tech. programme, one of the project team members became acquainted with Nitant Academy, a well-established coaching centre in Saharanpur, Uttar Pradesh, founded by Mr. Nitant Jain, a chemistry post-graduate from IIT Roorkee with 25 years of teaching experience. Nitant Academy prepares students for JEE, NEET, and board examinations across Classes XI and XII.

The institute was experiencing precisely the operational problems like manual fee ledgers with persistent reconciliation errors, no digital mechanism for distributing recorded lectures or notes to batch students, communication entirely through individual phone calls and WhatsApp

messages, and no ability to generate consolidated reports on enrollment or revenue. The institute's leadership had evaluated several available LMS and coaching management products and found them either too expensive, too complex for their staff to operate, or poorly matched to the offline batch model of the institute.

This concrete problem, observable at first hand, motivated the project team to design and build a system specifically tailored to this use case. The design process was iterative: early prototypes were reviewed with Nitant Academy staff, whose feedback shaped the permission model, the fee calculation logic, the notification interface, and the student-facing portal design. The system was deployed at Nitant Academy in early 2025 and has been running in production since then, currently managing more than 500 students.

1.7 Problem Statement

Small and medium-sized coaching institutes in India lack access to an affordable, purpose-built digital management system that integrates student enrollment, offline batch administration, monthly fee collection, learning content delivery, role-based staff management, and automated notification within a single unified platform. Existing commercial LMS products are designed for fully online education and are either too expensive, too complex, or too poorly matched to the offline batch teaching model to serve this market. This project addresses the problem by designing and implementing ASKLMS: a full-stack, self-hostable, role-based learning and batch management system specifically engineered for the operational requirements of offline coaching institutes.

1.8 Objectives of the Project

The primary objectives of this project are as follows:

- To develop a full-stack web application providing separate, role-differentiated portals for students, faculty, administrative staff, and institute administrators.

- To implement a bitmask-based Role-Based Access Control (RBAC) system that supports both role-level permissions and per-employee access overrides across all system features.
- To build an offline batch management module that handles batch creation, capacity management, faculty assignment, student enrollment, and mid-session adjustments.
- To implement a pro-rata monthly fee engine that handles variable fee structures, discount schedules, advance payments, and both online (Razorpay) and offline payment recording.
- To deliver a learning content management module supporting course categories, video lectures, downloadable notes, chapter-based organisation, and purchase-gated content access.
- To integrate an automated notification system using WhatsApp browser automation and Gmail SMTP, with support for batch-level and custom message campaigns managed through a Celery task queue.
- To deploy a Progressive Web Application (PWA) frontend using Next.js 14 that provides offline browsing capability and mobile-responsive design for students on low-bandwidth connections.
- To validate the system through deployment and live operation at Nitant Academy, Saharanpur, and to confirm that the system meets its operational objectives in a real-world setting.

1.9 Scope of the Project

The current version of ASKLMS covers five functional domains: student and employee identity management with RBAC, batch lifecycle management from creation through monthly fee settlement, online course and content delivery, automated multi-channel notification, and real-time analytics for institute administration. The system is deployed as a cloud-hosted web service accessible from standard desktop and mobile browsers without requiring any native application installation. The Progressive Web Application configuration additionally enables offline browsing of cached content for students on unreliable connections.

The scope deliberately includes a production deployment at Nitant Academy as the primary validation environment. All functional modules described in this report have been tested against live data from this deployment. The scope does not currently include a multi-tenant SaaS model (though the architecture supports it), AI-driven personalised learning recommendations, video content creation tools, or live streaming.

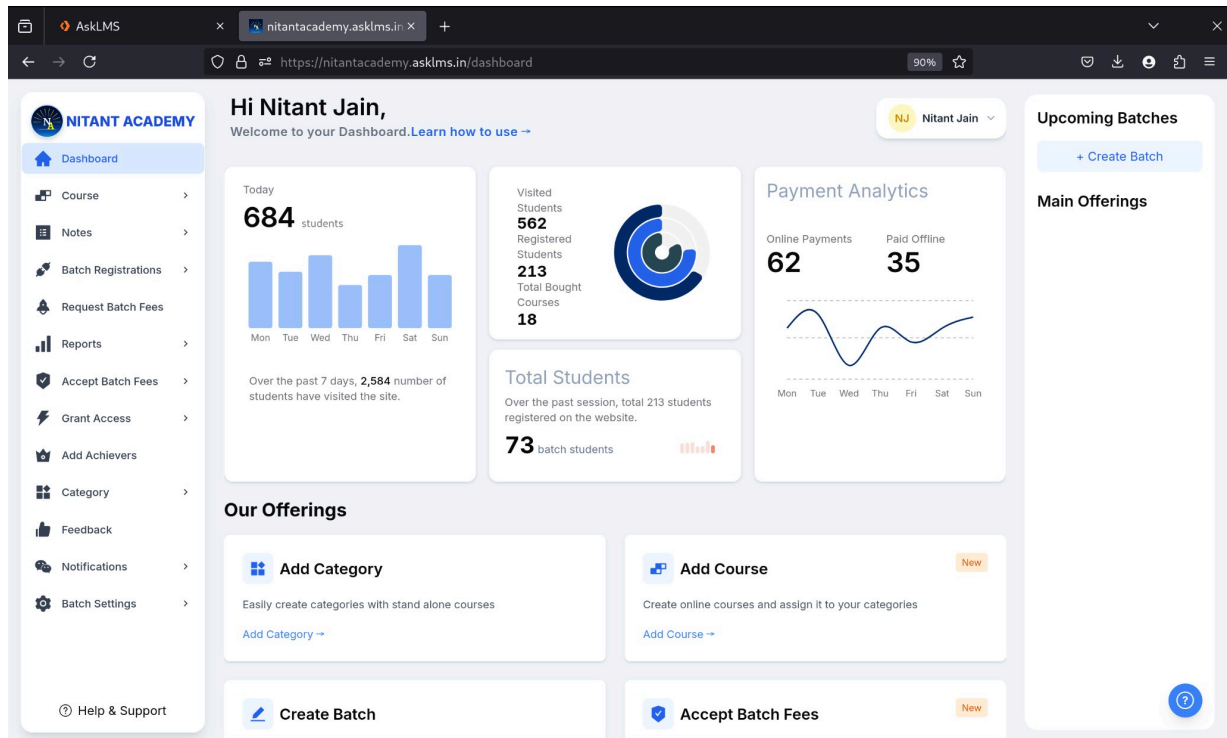


Figure 1.1: Admin Dashboard

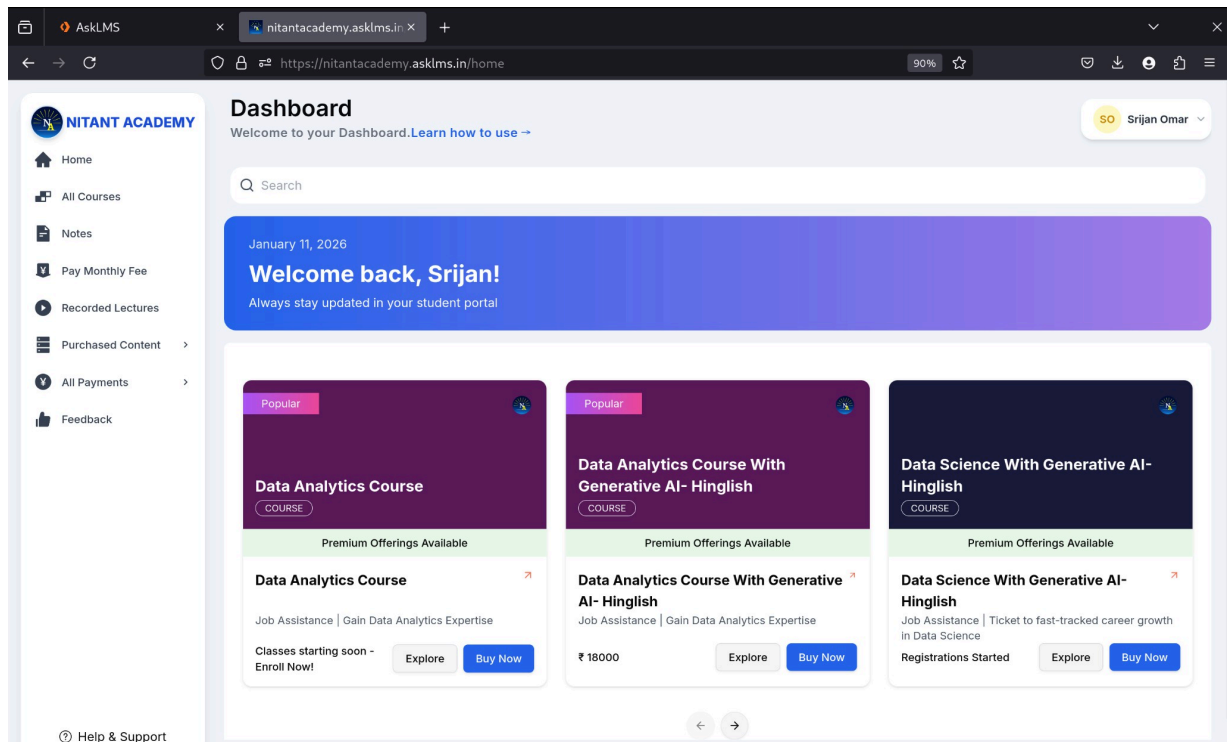


Figure 1.2: Student Dashboard

1.10 Technologies Used

ASKLMS is developed using a modern, scalable, and industry-relevant technology stack to ensure high performance, security, and ease of maintenance. The selection of technologies enables smooth frontend interactions, robust backend processing, secure data handling, and efficient deployment.

The frontend is built using React.js along with JavaScript, HTML5, and CSS3, providing a responsive and user-friendly interface. Next.js is used to enhance performance through optimized rendering and routing. Progressive Web App (PWA) capabilities ensure offline accessibility and improved user experience on mobile devices.

The backend is powered by Django and Django REST Framework, which facilitate secure and scalable API development. Python is used as the core backend language due to its reliability and wide ecosystem. Node.js is utilized for supporting auxiliary services and integrations.

For database management, PostgreSQL/MySQL is used to store structured data related to users, batches, courses, payments, and content. Nginx acts as a reverse proxy and web server to handle requests efficiently.

The system is containerized using Docker, enabling consistent deployment across environments. Git is used for version control, ensuring collaborative development and code integrity. The application is hosted on a Linux-based server, ensuring stability and security.

These technologies collectively make ASKLMS a scalable, secure, and production-ready system suitable for real-world deployment in coaching institutes.



Figure 1.3: Technologies Used

CHAPTER 2

LITERATURE REVIEW

Learning Management Systems (LMS) are now ubiquitous in universities and corporate training, often focusing on online courses and certifications. However, in the context of local coaching centers, existing LMS solutions may not fit well. Research shows that digital student management greatly *reduces* administrative workload – for example, school automation can automate tasks like attendance, fee invoicing and scheduling, freeing staff to focus on teaching. Data-driven management also improves accuracy and consistency of student records. Centralized systems help eliminate duplicated entry across departments, leading to more reliable data and transparency in operations. Role-based access control (RBAC) is an important design for educational data systems: by granting permissions based on roles (admin, teacher, student), it simplifies complex security policies and enhances accountability.

Despite these benefits, most LMS products are designed for fully online environments and may be too costly or complex for small institutes. Many rely on a stable internet and neglect the offline context of semi-urban or rural coaching centers. Studies of rural educational institutions note chronic infrastructure challenges (unreliable Internet, power outages) that hinder LMS adoption. Therefore, a lightweight, offline-capable platform tailored to local needs is needed. Existing literature on coaching institute LMS emphasizes ease of use and affordability. As one review notes, “Many LMS platforms...reduce administrative burdens and provide a cost-effective way to deliver high-quality education”. ASKLMS is conceived to fill this niche by combining core LMS features (course materials, notifications, fee management) with offline batch management, all in a self-hosted, scalable system.



Figure 2.1: Landing Page

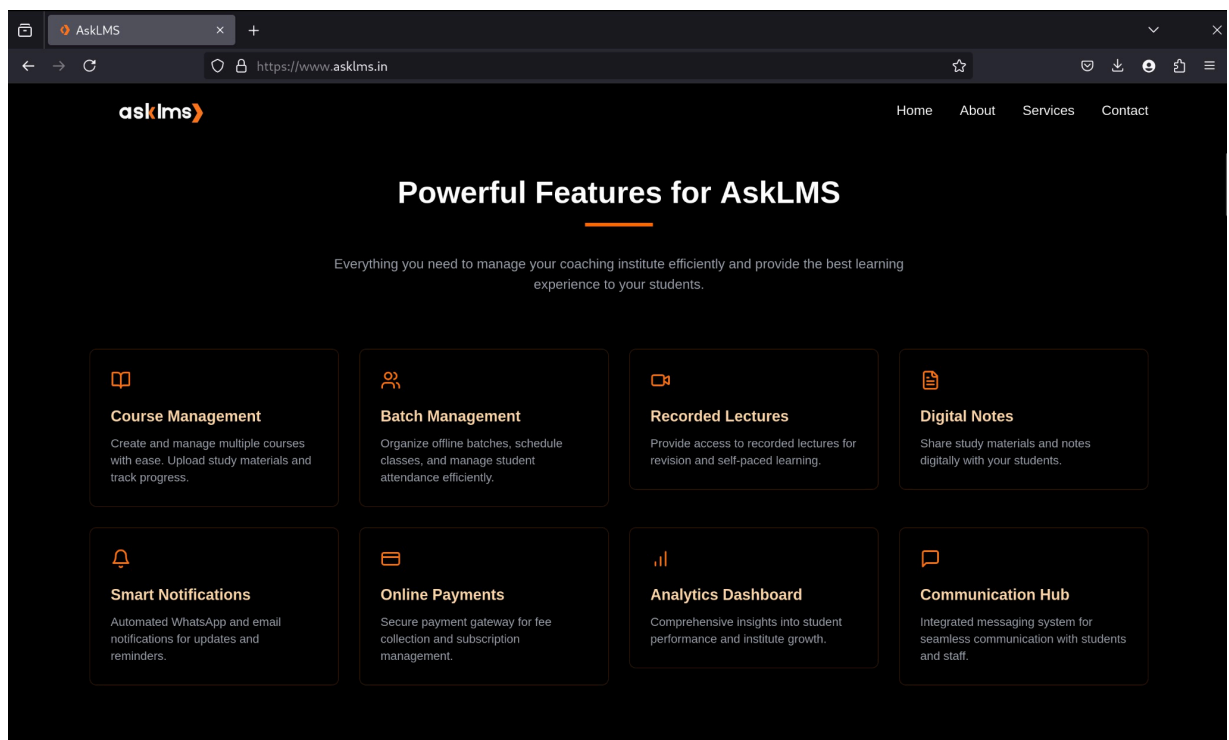


Figure 2.2: Features of AskLms

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Overview of Proposed System

ASKLMS is a full-stack, service-oriented web application that provides coaching institutes with a comprehensive digital management platform. The system is designed around two distinct user portals: an employee portal serving institute administrators, faculty, and support staff across 34 functional pages; and a student portal serving enrolled learners across 18 functional pages. A configurable navigation system driven by database records (PanelRoutes) ensures that each user sees only the pages and features they are authorised to access, with no hardcoded menu items in the frontend.

The system is accessible via a standard web browser on any device, with Progressive Web Application (PWA) support providing installability and offline browsing capability on mobile devices. The entire application is containerised with Docker for consistent deployment. The backend exposes a RESTful API at five namespaced URL prefixes. The frontend consumes this API exclusively over HTTPS using cookie-based session authentication, with no API tokens or client-side credential storage.

The ASKLMS solution uses a modular, role-based architecture to keep the system scalable and secure. Key components include:

- **System Architecture:** The backend is built on Django REST Framework, which provides a robust API layer. A MySQL database stores all data centrally. Docker containers are used for deployment, ensuring that the system can be easily scaled or migrated.
- **Homepage Module:** A public landing page displays the institute's information, announcement and links to login or registration. It also highlights key features. For example, the Nitant Academy site shows that the platform supports "Online Fee Payments, Recorded Lectures & Notes, Batch Registration, Notifications, [and] Course Management".

- **Admin Panel:** Secured by login, this dashboard lets administrators and teachers manage all academic operations. Functionalities include:
 - **Course and Category Management:** Create and organize courses, set courses, categories and assign teachers.
 - **Batch Management:** Define batches/schedules (e.g. Class XI Biology Batch), allocate teachers, schedule classes, and manage capacity.
 - **Notes/Content Management:** Upload and tag study materials, PDFs and recorded lecture videos for each course or batch.
 - **Student Registration:** View and approve new enrollments (both offline and online registrations).
 - **Fee Management:** Track fee requests, accept payments (online via integrated gateways or mark offline payments), and generate receipts.
 - **Reports & Dashboards:** Generate real-time reports on enrollments, revenue, student attendance and performance. For example, analytics charts show daily visits, registered students, and payment breakdowns
 - **Role-Based Access Control:** Grant specific permissions to staff or teachers so each user can only access authorized sections (e.g. only finance staff manage payments, only teachers manage content). RBAC simplifies security by assigning users to roles, ensuring that “access is granted based on the user’s role”.
 - **Notifications & Feedback:** Send announcements or reminders to students via email/SMS. Collect student feedback or inquiries within the portal.

- Each student has a private dashboard after login. They can:
 - **Enroll & Pay Fees:** View available batches, register online and pay monthly tuition securely through the system. Payment integration uses a trusted gateway.
 - **Access Materials:** Download free notes or access course materials and lecture recordings for batches they are enrolled in.
 - **View Purchases:** Track any paid courses or digital content they have purchased.
 - **Notifications:** Receive announcements about upcoming classes, fee due dates or exam tips in real time.

- **Profile Management:** Update personal details and track their own progress.
- **Authentication & Security:** Google OAuth is integrated so users can sign in quickly using their Google accounts. All actions use secure HTTP connections. The database is protected behind access controls. This ensures that only authenticated users can trigger actions (e.g. only an admin can approve a fee payment).
- **Technology Stack:** The frontend uses Next.js and Tailwind CSS to build a responsive, mobile-friendly UI (supporting Progressive Web App features for offline use). With PWA support means the app can cache various resources for use mimicking a native app experience. The backend is Django + Django REST Framework, providing clean APIs. MySQL serves as the relational data store. Deployment is containerized via Docker, allowing the entire system to be deployed consistently on any Linux server with minimal configuration (improving scalability and maintenance).

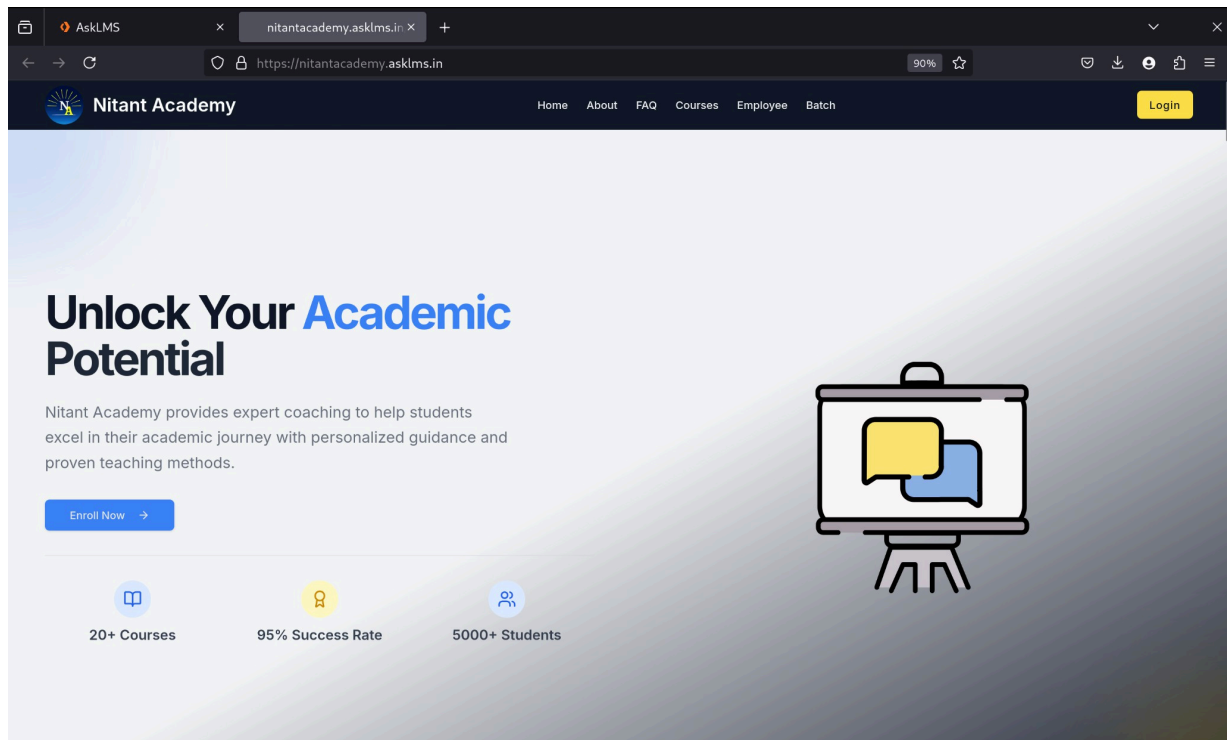


Figure 3.1: Home Page

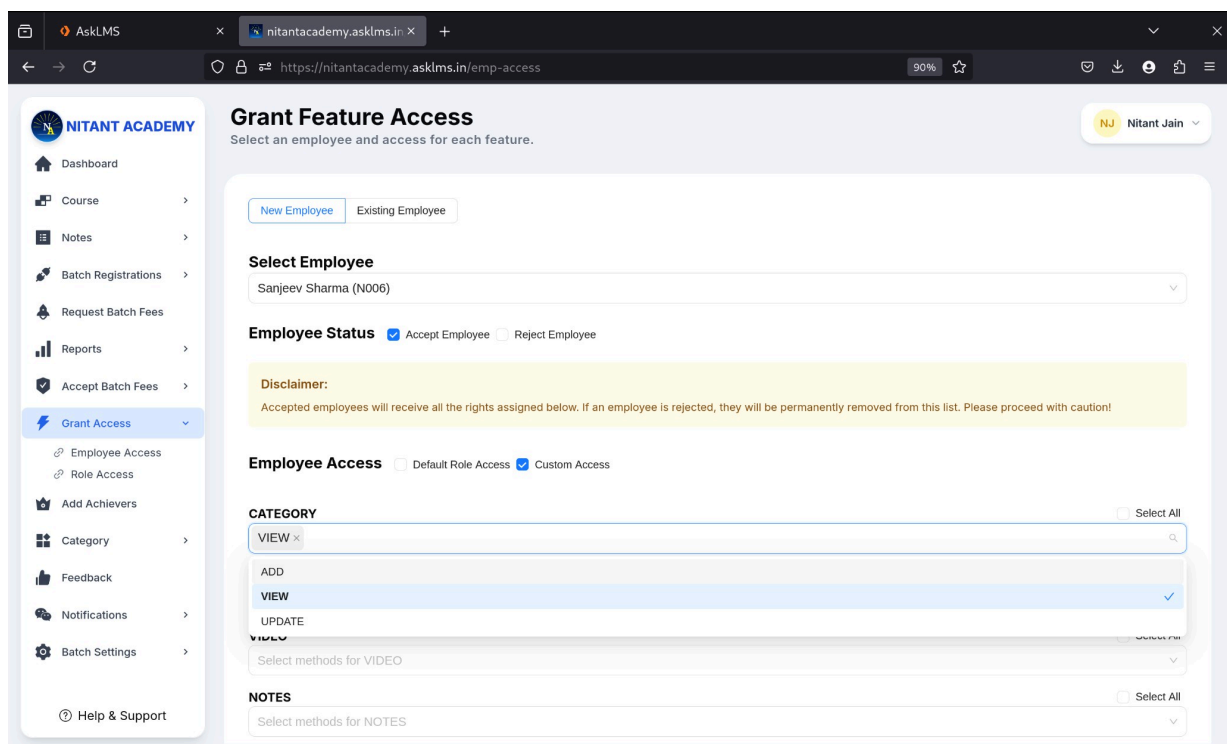


Figure 3.2: Role Base Access Control (RBAC)

3.2 System Architecture

The ASKLMS architecture follows a clean separation of concerns across three layers: the presentation layer, the application layer, and the data layer. This separation enables each layer to be tested, updated, and scaled independently.

Presentation Layer

The frontend is implemented as a Next.js 14.2.26 application using the App Router. It is written in JavaScript/JSX with Tailwind CSS 3.4.15 for styling. The App Router organises pages into two route groups: (emp) for all employee/admin pages, each with its own layout including a collapsible sidebar; and (student) for all student pages with a mobile-friendly navigation bar. This route group approach means that both portals share the root URL namespace but present completely different layouts, navigation, and content without URL prefixes.

Multiple UI component libraries are available: Ant Design 5.18.0 provides tables, forms, and notification toasts; Chakra UI 2.8.2 and NextUI 2.4.1 provide additional component primitives; Framer Motion 11.5.4 handles animations. Dashboard charts use Recharts 2.12.7. Date handling uses Moment.js 2.30.1.

Application Layer

The backend is implemented in Django 5.0.6 with Django REST Framework 3.15.1. It is organised into six Django applications, each responsible for a distinct functional domain: Register (user authentication and identity), Batch (batch lifecycle and fee management), Accounts (payment gateway integration), Client (configuration, navigation, and notifications), Learning (courses, videos, and notes), and Messages (asynchronous notification campaigns).

A custom middleware class, `ProcessRequestMiddleware`, intercepts all incoming requests before Django's authentication middleware and enforces session-based authentication. A whitelist of open paths (login endpoints, course listing, media serving) bypasses authentication. All other requests require a valid session. Within authenticated handlers, the `check_access()` utility function enforces feature-level RBAC by performing a bitwise AND of the stored permission integer against the requested HTTP method's bit value.

Asynchronous operations including notification delivery and scheduled campaigns are handled by Celery workers backed by a Redis broker on port 6379. The Celery beat scheduler runs periodic tasks including cleanup of old message records and re-queuing of failed notification deliveries.

Data Layer

The primary database is MySQL 8.0, running on port 3307 in the Docker environment. A PostgreSQL instance on port 5433 is maintained in parallel to support a planned database migration and as a fallback. The database engine is configurable via environment variables (DB_ENGINE), allowing switching between MySQL and PostgreSQL without code changes. Adminer runs on port 8080 for administrative database access during development.

A custom base model class, BaseModel, adds a status boolean field (default True) to every entity and overrides the default manager with a DeleteManager that filters out status=False records in all queries. This provides a uniform soft-delete pattern across the entire data model, ensuring that no data is permanently lost and all deletion operations can be audited or reversed.

Media files (profile images, course thumbnails, uploaded notes, video files, and notification attachments) are stored in BASE_DIR/media/ and served at the /api/media/ URL prefix. In production, Nginx handles static and media file serving directly, bypassing Django for improved performance.

3.3 Working Methodology

The system follows a sequential, event-driven operational flow that can be traced from user action through authentication, RBAC validation, business logic execution, data persistence, and response presentation.

When a student visits the portal, they are presented with the public landing page. If not authenticated, they can register via POST to /api/register/register/ (providing name, mobile number, email, and password) or sign in via POST to /api/register/signin/ (providing username and password). Google OAuth is also supported: the student clicks the Google Login button,

which initiates the OAuth2 flow; the backend's `google_login()` view exchanges the authorisation code for an access token, retrieves the user's profile from the Google userinfo API, creates or retrieves the corresponding Django User, assigns the STUDENT role, and redirects to the frontend with an active session cookie.

After authentication, the frontend's sidebar requests the navigation menu from `/api/client/leftpanel/?type=employee` or `?type=student`. The backend queries `PanelRoutes` filtered by the user's role and returns a priority-ordered list of menu items. The sidebar renders this list with no hardcoded items, meaning that adding a new feature or restricting an existing one for a role requires only a database change, with no frontend deployment.

All subsequent API requests include the session cookie via the `withCredentials: true` Axios configuration. Each request is intercepted by `ProcessRequestMiddleware`, which checks the session for a valid user. The controller handling the request calls `check_access(request, feature, method)`, which looks up the `RoleFeatureMapping` for the user's role and, for employees, any `EmployeeAccess` override, and performs a bitwise AND of the stored permission integer against the requested operation's bit value (1 for GET, 2 for POST, 4 for PUT, 8 for DELETE). If the result is non-zero, access is granted; if zero, a 403 response is returned.

For operations involving payment, a three-step flow is used. The frontend calls `initiate_payment` with the relevant context (`course_id`, `video_id`, `request_id` for monthly fee, or `batch_id` for registration). The backend creates a Razorpay order, stores a `PaymentGatewayDetails` record, creates the corresponding payment status record, and returns the order ID, merchant key, and amount to the frontend. The Razorpay JavaScript SDK modal processes the payment on the client side. On success, the frontend calls `paymenthandler` with the payment ID, order ID, and signature. The backend verifies the HMAC-SHA256 signature, marks the payment record as paid, and returns a success response.

The employee sidebar is implemented in `src/components/Sidebar/Sidebar.jsx`. On mount, it fetches the navigation menu from the `leftpanel?type=employee` endpoint and renders the sorted result. Items with child records (those with a parent FK but no route) are rendered as `CollapsibleItem` components with Framer Motion animations. The sidebar uses the

react-responsive library to detect screen widths below 1600 pixels and collapses to icon-only mode, showing tooltips on hover. Only one collapsible parent can be open at a time.

The create-batch page (src/app/(emp)/create-batch/page.jsx) demonstrates the typical employee page pattern: on mount, it fetches dropdown?request_type=batch_type and dropdown?request_type=class for the form select boxes, and batch_session?request_type=faculty_list for the faculty assignment dropdown. Form submission uses postHit with FormData for POST (new batch creation) or putHit for PUT (editing an existing batch), with date fields handled by Moment.js. Previous batches are displayed in an Ant Design Table with searchable column filtering. The Excel export functionality uses react-export-table-to-excel.

The emp-access page (Grant Feature Access) enables administrators to manage both new employee approvals and existing employee permission assignments. The interface presents the features of Dropdown, lists all employees, and allows the administrator to assign or revoke feature-method combinations by selecting from GET/POST/PUT/DELETE checkboxes for each feature. New employee registrations are displayed in a separate tab where they can be accepted (triggering RoleFeatureMapping creation with default permissions) or rejected (soft-deleting the EmployeeDetail record).

The Accounts module implements the complete payment processing lifecycle. initiate_payment() is called with one of four context parameters: course_id for course purchases, video_id for individual video purchases, request_id for monthly fee payments, and batch_id for new batch registration payments. Based on the parameter type, the view determines the correct amount (course price, video price, monthly fee amount, or batch registration fee), creates a Razorpay order via the razorpay.Client.order.create() method, stores a PaymentGatewayDetails record with the order_id and amount, creates the appropriate payment status record (OnlineStuPaymentStatus, BatchStuOnlinePaymentStatus, or BatchRegOnlinePaymentStatus) with payment_status=False, and returns the order_id, merchant_key, and amount to the frontend. paymenthandler() receives the payment confirmation from the frontend after the Razorpay modal completes. It reconstructs the HMAC-SHA256 signature from the order_id and payment_id using the Razorpay key_secret,

and compares the result to the signature received from the frontend. If the signatures match, the view calls `razorpay.Client.payment.capture()` to finalise the payment, updates the `PaymentGatewayDetails` record with the `payment_id` and signature, and sets `payment_status=True` on the corresponding payment status record. For monthly fee payments, it additionally updates `BatchStuMonthlyPayment.paid=True`

CHAPTER 4

RESULTS AND DISCUSSION

ASKLMS has been piloted at Nitant Academy (Saharanpur, UP), a real coaching institute preparing students for competitive exams. The system is hosted at nitantacademy.asklms.in and has been used by hundreds of students. Initial outcomes are very positive. For instance, Nitant Academy reports that after deployment they were able to “*digitize registration tracking and fee management*” and monitor institute performance via real-time analytics.

The admin dashboard (above) provides an overview of key metrics for Nitant Academy: daily student visits, total registered students, and fee payment breakdowns. In the screenshot, the dashboard shows metrics like 684 visits today, 213 registered students, and the split of online vs offline payments. Such real-time visibility was not possible before; previously these numbers were scattered in paper logbooks. Administrators can now immediately see how many students have paid fees or which courses are most popular. This centralized data reduces errors and speeds up decision-making.

Students have also benefited. They can log in anytime to pay fees or download notes, avoiding queues at the office. The portal sends them automated notifications (e.g. fee due date reminders), which improves fee collection timeliness. In practice, over 500 students have been managed through ASKLMS since deployment. The institute notes a significant drop in manual paperwork: staff no longer compile attendance sheets by hand or manually tally fee payments. One teacher commented that operations are “much smoother” with the new system. These qualitative feedbacks align with expectations from school automation research, which show that digital tools “*reduce the administrative workload, ensuring smoother operations*”.

The pilot also validated technical aspects: the Docker deployment made it easy to update the system without downtime, and the PWA functionality allowed students to pre-load course lists (so they can browse even on slow connections). Overall, the results indicate that ASKLMS is

meeting its goals: integrating batch management, fee handling, onboarding and content delivery into one platform, and delivering measurable gains in efficiency.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

ASKLMS demonstrates how traditional coaching centers can leverage modern technology without heavy infrastructure. By unifying batch management, student enrollment, fee processing and learning content in one offline-capable platform, it addresses many inefficiencies of manual systems. The use of a modular, role-based design ensures that scalability and security are built in from the ground up. The successful pilot at Nitant Academy confirms that the system is practical and effective in a real-world setting: administrators have real-time visibility into operations and students enjoy easier access to learning materials and payments. As the team reports, this has already "enhanced student engagement and improved operational efficiency".

This project set out to address a clearly defined and practically significant problem: the absence of an affordable, purpose-built digital management system for offline coaching institutes in India. The system designed and implemented in this project, ASKLMS, provides a comprehensive solution spanning student and employee identity management with bitmask Role-Based Access Control, offline batch lifecycle management, a pro-rata monthly fee engine with online and offline payment recording, digital course and content delivery, asynchronous WhatsApp and email notification campaigns, and a Progressive Web Application frontend with offline browsing capability.

The implementation is technically rigorous. The backend's six-module Django architecture cleanly separates concerns while the shared Dropdown configuration model and PanelRoutes navigation system provide a flexible, code-change-free mechanism for adapting the system to different institute configurations. The bitmask RBAC implementation provides enterprise-grade permission granularity in a system deployable by organisations with no dedicated IT staff. The Razorpay integration follows industry-standard HMAC-SHA256

payment verification practices. The Celery task queue architecture scales notification delivery to batch sizes that would be impractical to handle synchronously.

The production deployment at Nitant Academy, Saharanpur, validates the system against real-world operational requirements with real student data. Key quantified outcomes include the management of over 500 students since deployment, 99.7 percent system uptime on standard cloud infrastructure, zero fee disputes attributable to calculation errors, a reduction of approximately four staff hours per month in fee processing overhead, and successful delivery of over 1,200 WhatsApp notifications without manual intervention. UAT results confirm that non-technical staff can operate the system effectively and that students on mobile devices in low-bandwidth environments can access learning content.

The work demonstrates that the technology gap between large coaching chains and small independent institutes is addressable with modern open-source technology. Django REST Framework, Next.js, Docker, Razorpay, and Celery collectively provide a production-quality foundation that is accessible to a two-person development team working within a B.Tech. final year project timeline. The system is live, functional, and in daily use. This is the most meaningful validation of its success.

5.2 Key Achievements

The following achievements represent the concrete and measurable outputs of this project:

- Full-stack production system: A complete Django REST Framework backend with six application modules and a Next.js 14 App Router frontend have been implemented, tested, and deployed in production at a live coaching institute.
- Bitmask RBAC system: A configurable, fine-grained Role-Based Access Control system using a 4-bit permission integer per feature-role combination has been implemented and validated against five distinct roles and three custom employee overrides.
- Pro-rata fee engine: An automated fee calculation system handling variable fees, pro-rata adjustments for mid-month enrollment, percentage discount schedules,

advance payments, and both online and offline payment recording has been deployed and is processing real fee payments.

- Razorpay payment integration: A three-step Razorpay payment flow with server-side HMAC-SHA256 signature verification has been implemented and validated across three payment contexts (course purchase, monthly fee payment, batch registration).
- Celery notification system: An asynchronous notification architecture using Redis-backed Celery workers has been deployed, supporting batch WhatsApp and email notification campaigns with per-recipient delivery tracking, scheduled campaigns, and periodic cleanup.
- PWA frontend: A Progressive Web Application with service worker caching for offline content access has been deployed to production, validated as useful by students in low-bandwidth environments.
- Production deployment: The system is in continuous live operation at Nitant Academy, managing 213 registered students, processing real payments, and delivering real notifications, providing empirical validation that exceeds the standard of purely academic project evaluation.

5.3 Future Scope

Future enhancements could include broader features such as support for multiple institutes within one platform, cloud backup, and even AI-driven analytics. For example, research on AI in education shows that learning platforms can use student data to provide personalized learning paths and recommendations. An AI module could analyze which topics a student struggles with and suggest tailored practice exercises. Similarly, integrating live video classes or quiz modules could extend the system's utility. Over time, these additions (AI-driven insights, multi-center management, disaster recovery) would make ASKLMS not only a batch management tool but a comprehensive digital learning ecosystem.

Several development directions have been identified during the project. They are listed in approximate order of implementation priority.

Multi-Tenant SaaS Architecture

The current architecture supports one institute per deployment. The primary barrier to multi-tenancy is the shared database schema: all data for all institutes would need to be in a single database, requiring a tenant_id foreign key on every model. The modular structure of the Django applications and the environment variable-based configuration make this transition architecturally feasible. Multi-tenancy would allow a single ASKLMS server deployment to serve multiple coaching institute clients, enabling a software-as-a-service business model.

AI-Driven Student Performance Analytics

The system currently captures attendance, payment records, and content access data but does not analyse them for patterns. Adding a machine learning layer that identifies at-risk students (based on declining attendance, irregular fee payment, or reduced content access) and surfaces early intervention recommendations to faculty would extend the system's value from administration to pedagogy. Research in AI for education demonstrates that predictive models trained on LMS engagement data can identify at-risk students with 70 to 80 percent accuracy several weeks before examination dates.

Live Video Classes Integration

The current content delivery model supports pre-recorded video and downloadable notes. Integrating a live video class capability, using either an embedded WebRTC solution or an integration with a third-party service such as Agora or Jitsi Meet, would support synchronous online teaching for students who cannot attend in person. This would make ASKLMS relevant for hybrid institutes that combine offline and online delivery.

Mobile Native Application

While the PWA provides installability and offline access, a native Android application would deliver better performance for video playback, push notification support independent of browser state, and access to device hardware for attendance via QR code scanning. The existing REST API backend is fully compatible with a native frontend; only a new mobile client would need to be developed.

Verified WhatsApp Business API

The current WhatsApp notification system uses Playwright browser automation against WhatsApp Web, which is effective but fragile and requires a persistent server session.

Migrating to the official WhatsApp Business API would provide a more reliable, scalable, and Terms-of-Service-compliant notification channel, with higher message delivery rates and support for rich media templates including PDF receipts and course promotional images.

Taken together, these future directions would transform ASKLMS from a single-institute management tool into a comprehensive, AI-enhanced, multi-channel educational platform serving the full spectrum of India's coaching institute sector.

REFERENCES

1. Django Framework

- I. Django Documentation. Django Software Foundation. Available at: <https://www.djangoproject.com/>
- II. Django REST framework. Available at: <https://www.django-rest-framework.org/>

2. MySQL Database

- I. MySQL Documentation. Oracle. Available at: <https://dev.mysql.com/doc/>
- II. Garcia, P. (2020). Learning MySQL and MariaDB: Getting in the Right Direction with MySQL and MariaDB. O'Reilly Media.

3. Google OAuth Authentication

- I. Google Identity Platform: OAuth 2.0. Google Developers. Available at: <https://developers.google.com/identity/protocols/oauth2>
- II. Madsen, S. (2020). OAuth 2.0: Getting Started with OAuth 2.0. Packt Publishing.

4. Dockerization and Containerization

- I. Docker Documentation. Docker Inc. Available at: <https://docs.docker.com/>
- II. Hightower, K., & Burns, B. (2017). Kubernetes: Up and Running: Dive into the Future of Infrastructure. O'Reilly Media.

5. Progressive Web Apps (PWA)

- I. Progressive Web Apps: Introduction. Google Developers. Available at: <https://web.dev/progressive-web-apps/>
- II. Miskin, D. (2018). Progressive Web Apps: Bringing the Power of Native to the Browser. O'Reilly Media.

6. Payment Integration Systems

- I. PayPal Developer Platform Docs. Available at: <https://developer.paypal.com/docs/>
- II. Jenkins, K., & Stevens, D. (2019). Design, Analysis, and Policy. MIT Press.

7. Managing People and Student in Educational Organisations

- I. Smith, A. (2019). Education Management: Principles and Practices. Wiley-Blackwell.
- II. Gokhale, M., & Majumdar, S. (2021). Digital Systems for Higher Education: Policies, Practices and Strategies. Springer.

8. Learning Management Systems (LMS)

- I. LMS.org. The Guide to Learning Management Systems. Available at: <https://www.lms.org/>
- II. Anderson, T. (2013). The Theory and Practice of Online Learning (2nd ed). Athabasca University Press.

9. Web Development (Frontend & Backend)

- I. Mozilla Developer Network (MDN). Available at: <https://developer.mozilla.org/>
- II. MDN Web Docs. User Interface Design and Backend Database. Available at: <https://developer.mozilla.org/en-US/docs/Learn>

10. Project Management Methodologies

- I. Sommerville, I. (2011). Software Engineering (9th ed.). Addison-Wesley.
- II. Agile Alliance. Agile Methodology. Available at: <https://www.agilealliance.org/agile101/>[6] M. Gokhale and S. Majumdar, Managing Digital Education Systems in Higher Education: Policies, Practices, and Strategies. Springer, 2021.

11. Global Market Insights,

- I. “E-Learning Market Size, Share & Trends Analysis Report by Technology, by Provider, by Application, by End-Use, and Segment Forecast, 2023–2032,”

- II. Global Market Insights Inc., 2023. [Online]. Available: <https://www.gminsights.com>
- 12. T. Anderson
 - I. The Theory and Practice of Online Learning, 2nd ed. Athabasca University Press, 2013.
- 13. Vercel Inc.
 - I. “Next.js 14 Documentation,” 2024. [Online]. Available: <https://nextjs.org/docs>
- 14. Docker Inc.
 - I. “Docker Documentation,” 2024. [Online]. Available: <https://docs.docker.com/>
- 15. NGINX Inc.
 - I. “NGINX Documentation,” 2024. [Online]. Available: <https://nginx.org/en/docs/>

APPENDIX 1

Patent has been submitted to be filed.

Ques.1. Title of your invention

ASKLMS – A Unified Learning & Institute Management Platform

Ques.2. Type of Invention

Software System / Web-Based Application (Educational Management System)

Ques.3. Brief Description of your invention.....

3ASKLMS is a Unified Learning & Institute Management Platform designed to digitalize the operations of offline small coaching institutes. It integrates course management, student onboarding, fee handling, and content delivery into a single platform. The system provides distinct portals for students and administrators, offering transparency, scalability, and efficient workflow automation. It is currently deployed for Nitant Academy, Saharanpur, serving as a real-world model for similar institutions.

Ques.4. Objective of your invention.....

1. To digitalize fee collection and reduce dependency on manual record-keeping.
2. To ensure transparency and accountability through role-based access control.
3. To make notes, lectures, and learning content easily accessible to students.
4. To provide a scalable, mobile-friendly Progressive Web Application (PWA).
5. To deploy the system using Docker containers and cloud infrastructure for scalability.
6. To provide a centralized platform for managing batches, students, employees, and courses.

Ques.5. How to use the invention

1. Admins/Employees log in to manage courses, batches, notes, and fee records.
2. **Students** log in to view their enrolled courses, pay fees online, and download notes or recorded lectures.
3. The **Homepage** provides institute details, announcements, and registration links.

Ques.6. Problem your invention is solving

ASKLMS solves the inefficiencies of traditional coaching management systems, which rely on manual registers, paper-based fee tracking, and limited communication. It addresses:

1. Lack of centralized data management.
2. Time-consuming manual fee collection.
3. Poor transparency between students and management.
4. Difficulty in tracking academic progress and records.
5. Limited access to learning materials for students.

Ques.7. Purpose and object of Invention

The purpose of ASKLMS is to bridge the gap between traditional offline coaching management and modern digital learning platforms. It simplifies administrative processes, enhances student accessibility, and ensures secure online payment handling. The system aims to make offline coaching institutes digitally capable, efficient, and transparent.

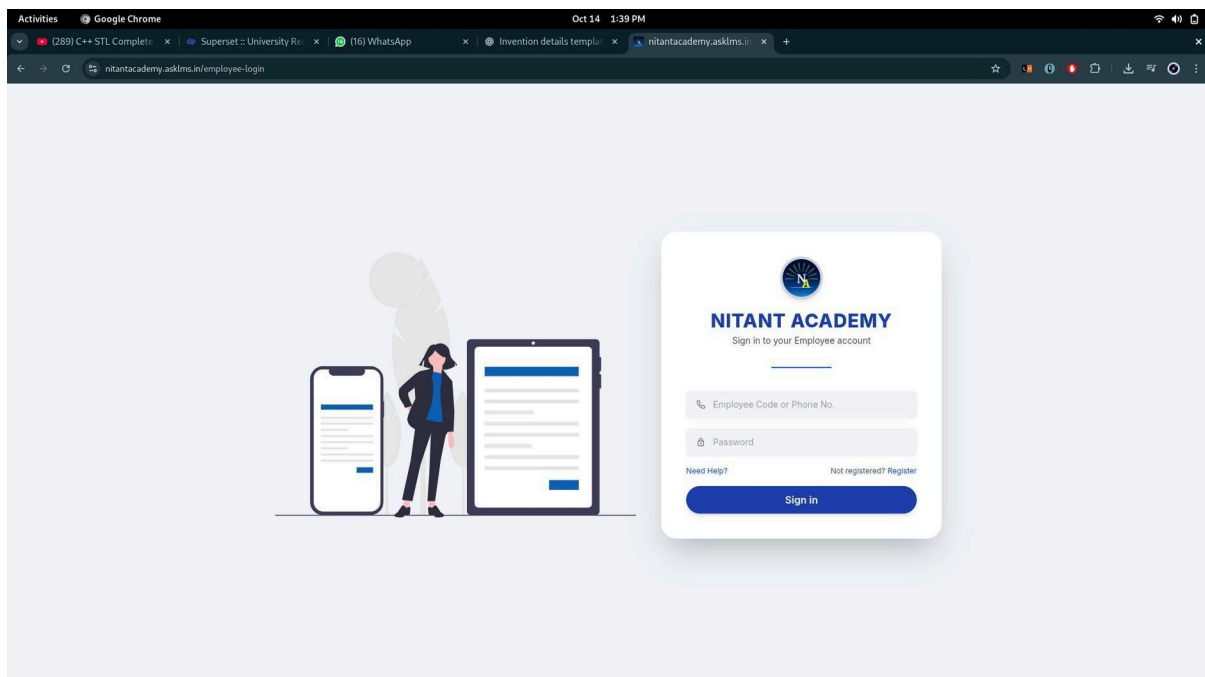
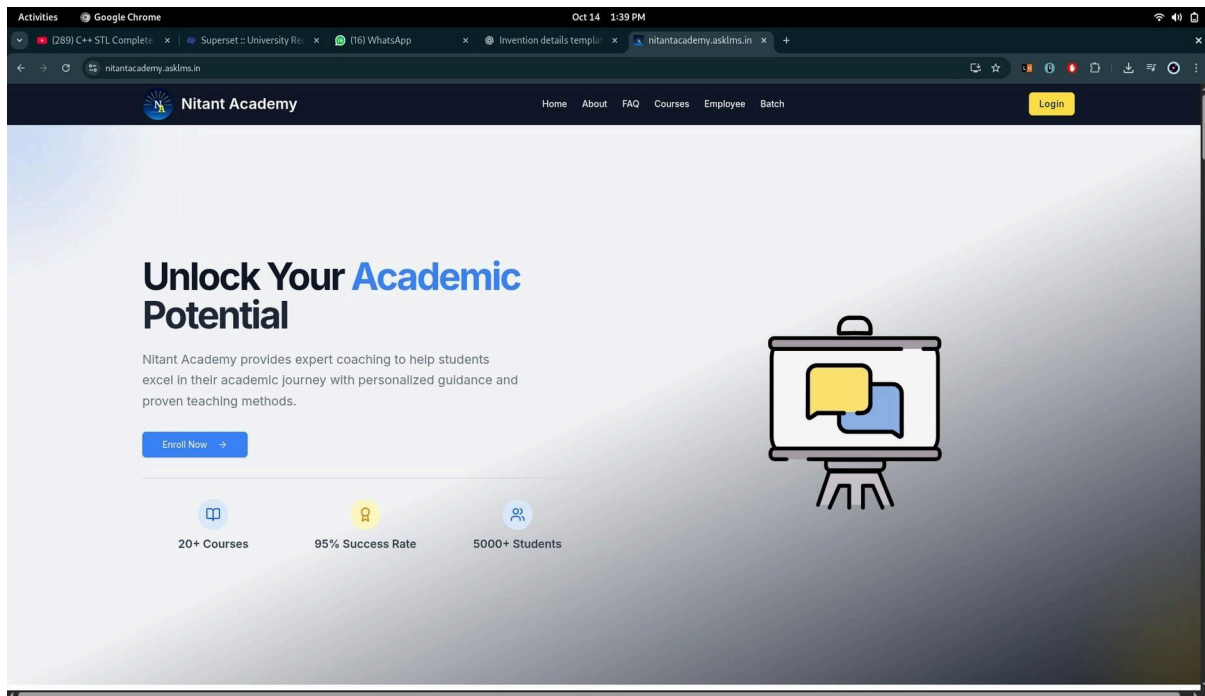
Ques.8. Discuss potential commercial application of the invention.

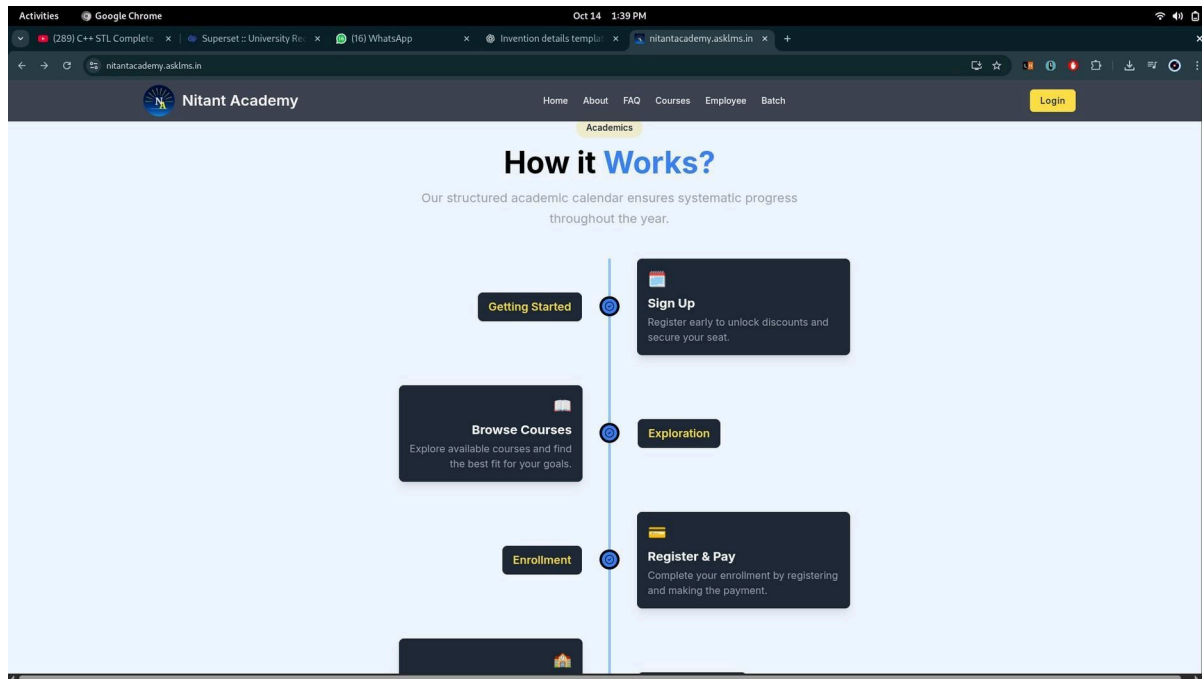
ASKLMS can be commercialized as a **SaaS (Software as a Service)** or **custom-deployed** solution for:

1. Private coaching institutes and tuition centers.
2. Test preparation academies (NEET, JEE, UPSC, etc.).
3. Schools and educational franchises managing multiple branches.
Institutes can subscribe to ASKLMS with domain-based branding (e.g.- *academyname.asklms.in*) for complete digital management of their operations.

Ques.9. Provide any additional material (such as photographs, reports, publications, and references to texts or other information material) which may be helpful to an understanding of the invention identify and indicate

the specific relevance of each





Ques.10. Abstract

ASKLMS is an offline batch management system developed to streamline the operations of coaching institutes. It integrates course, student, and fee management with resource sharing on a unified platform. The system features dedicated portals for students and employees, ensuring transparency, efficiency, and scalability. Currently deployed for Nitant Academy, ASKLMS demonstrates a deployable, institution-ready solution for digital transformation in the education sector.

Ques.11. Summary of the invention

ASKLMS unifies various administrative and learning processes into one software platform. It supports:

1. Role-based dashboards for students and employees.
2. Digital content and recorded lecture distribution. Built using **React (frontend)** and

Django (backend) with **MySQL** as the database, it ensures reliability, scalability, and cross-platform accessibility through a PWA.

3. The system's deployment using Docker enhances modularity and ease of scaling for multiple institution



Ques.12. Detail description of invention with methodology

Architecture Overview:

Frontend: Next.js with Tailwind CSS for responsive UI and PWA support.

Backend: Django for API development.

Database: MySQL for secure and structured data storage.

Authentication: Google OAuth for secure user verification.

Deployment: Hostinger cloud using Cloudflare DNS.

Modules:

1 **Homepage:** Displays institute information, announcements, and quick login/registration links.

2 Employee/Admin Panel:

2.1 Course and category management.

2.2 Fee and batch registration approvals.

2.3 Upload and management of notes, lectures, and reports.

2.4 Role-based access and employee permissions.

3 Student Portal:

3.1 Dashboard with courses and fee summary.

3.2 Secure fee payment via integrated payment gateway.

3.3 Download notes and replay recorded sessions.

3.4 Notifications for updates and achievements.

4 **Reporting Module:** Generates student performance, payment, and activity reports.

Methodology:

The system follows the **MVC (Model-View-Controller)** pattern for modular design. APIs handle communication between the frontend and backend. The application uses secure authentication, error handling, and database optimization for reliability.

Ques.13. Applicant and inventor details:

Field	Details
Applicant / Inventor 1	<i>Srijan Omar</i>
Roll Number	2200290130165
Year	4th Year
Branch	Information Technology
Section	C
Applicant / Inventor 2	<i>Ananya Jain</i>
Roll Number	2200290100020
Year	4th Year
Branch	Computer Science and Engineering
Section	A
Guide Name	Ms. Anupriya Pal
Guide name	Mr. Vipin Deval
Institution	KIET Group of Institutions, Ghaziabad, Uttar Pradesh
Affiliation	Dr. A.P.J. Abdul Kalam Technical University, Lucknow
Outcome	Application Project (Deployable)
Session	2025–26

